

RTL DESIGN & VERIFICATION

9x9 Tiled Matrix Multiplier

Systolic Array Architecture with APB
Interface on Artix-7 FPGA

PROJECT	matmul_9x9	TARGET DEVICE	xc7a35tcpg236-1 (Artix-7 35T)
DESIGN STYLE	Synthesizable RTL (Verilog + SystemVerilog TB)	INTERFACE	AMBA APB v2.0
SIMULATION TOOL	Vivado XSim 2024	VERIFICATION	10 test cases 810 elements checked

Systolic Array

Tiling

APB Slave

SRAM

Signed Arithmetic

3x3 PE Array

1. Introduction

This project implements a complete hardware accelerator for multiplying two 9×9 signed integer matrices, targeting a Xilinx Artix-7 FPGA. The design uses a 3×3 systolic processing-element (PE) array and a tiling controller that decomposes the full 9×9 operation into nine 3×3 tile-multiplication passes. Data is stored in a shared on-chip SRAM and loaded/read by a host through an AMBA APB slave interface.

Design goals: fully synthesizable RTL, no external memory, signed 32-bit operands, pipelined systolic throughput, and a clean APB interface for software programmability.

1.1 Problem Statement

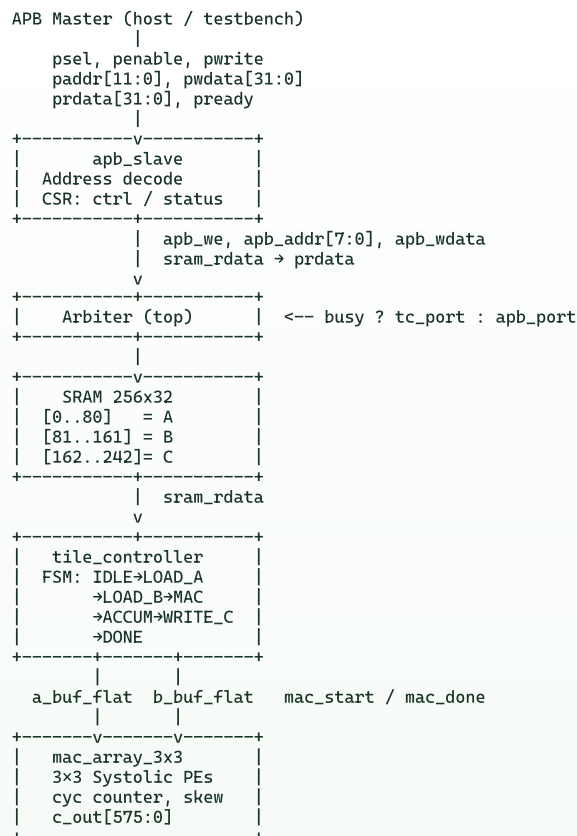
Compute $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ where A, B, C are 9×9 matrices of 32-bit signed integers. Naive hardware would require 81 multipliers running simultaneously. The tiled approach reuses a compact 3×3 PE array nine times, dramatically reducing area while achieving the same result in bounded latency.

1.2 Key Design Decisions

- **3×3 systolic PE array** — weight-stationary, diagonal wavefront; only nearest-neighbor wires
- **Single SRAM** — one 256-deep $\times$ 32-bit synchronous SRAM stores A (0–80), B (81–161), C (162–242)
- **APB interface** — maps SRAM regions and a control/status register to a 12-bit address space
- **Tiling loop** — FSM-driven tile controller iterates over all 27 tile pairs, accumulates partial products

2. System Architecture

2.1 Top-Level Block Diagram



2.2 Address Map

Base Address	Region	Size	Access	SRAM Offset
0x000	Matrix A	81 × 32-bit words	R/W	0 – 80
0x200	Matrix B	81 × 32-bit words	R/W	81 – 161
0x400	Matrix C (result)	81 × 32-bit words	R only	162 – 242
0x600	Control register	1 word	W only	—
0x604	Status register	1 word	R only	—

Control register bit [0] = **start**. Status register bit [0] = **done**, bit [1] = **busy**.

3. Module Descriptions

3.1 pe.v — Processing Element

The fundamental compute unit of the systolic array. Each PE is a registered multiply-accumulate cell that also passes its operands to the right (A) and downward (B), forming the systolic pipeline.

Port	Direction	Width	Description
clk, rst_n	in	1	Clock / active-low reset
clear	in	1	Synchronously zero c_acc (start of new tile)
a_in, b_in	in	DW=32	Operands from left / top neighbor
a_out, b_out	out	DW=32	Pass-through to right / bottom neighbor
c_acc	out	AW=64	Running signed accumulator

```
// Registered pass-through + signed MAC
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    a_out <= 0; b_out <= 0; c_acc <= 0;
  end else begin
    a_out <= a_in; b_out <= b_in;
    if (clear) c_acc <= 0;
    else c_acc <= $signed(c_acc) + $signed(a_in) * $signed(b_in);
  end
end
```

3.2 mac_array_3x3.v — Systolic MAC Array

Instantiates a 3×3 grid of PEs and implements the diagonal-wavefront skewing that ensures A[i][k] and B[k][j] arrive at PE[i][j] in the same clock cycle.

SYSTOLIC TIMING — WAVEFRONT SKEWING

Cycle counter: cyc = 1 (clear), 2..8 (data feed), 9 (done)

A row injection (left edge):
row 0: cyc = k+0+2 (2,3,4)
row 1: cyc = k+1+2 (3,4,5)
row 2: cyc = k+2+2 (4,5,6)

B col injection (top edge):
col 0: cyc = k+0+2 (2,3,4)
col 1: cyc = k+1+2 (3,4,5)
col 2: cyc = k+2+2 (4,5,6)

A[i][k] meets B[k][j] at PE[i][j] at cyc = k+i+j+2 ✓

Interconnect (nearest-neighbour only):
a_h[row*4+col] : row i, col 0 = left-edge input, col 1..3 = PE a_out chain
b_v[row*3+col] : row 0 = top-edge input, rows 1..3 = PE b_out chain

Parameter	Value	Notes
Tile latency	8 cycles	1 clear + 7 data cycles
Throughput	1 tile / 8 cycles	Back-to-back with clear pipeline
Interconnect	Local only	No broadcast wires; scales well
Accumulator width	64-bit signed	No overflow for 9×3×3 partial sums

3.3 `tile_controller.v` — Tiling FSM

Orchestrates the full 9×9 multiply as 27 tile-MAC operations (9 output tiles × 3 accumulation steps each). The FSM steps through states that load A and B sub-tiles from SRAM, fire the MAC array, and write partial-sum results back to SRAM in the C region.

FSM State	Action
ST_IDLE	Wait for <code>start</code> ; initialize tile indices br=0, bc=0, bk=0
ST_LOAD_A	Read 9 elements of A tile [br][bk] from SRAM into <code>a_buf</code>
ST_LOAD_B	Read 9 elements of B tile [bk][bc] from SRAM into <code>b_buf</code>
ST_MAC	Assert <code>mac_start</code> ; wait for <code>mac_done</code>
ST_READ_C / ST_ACCUM	Read existing partial sums from C SRAM, add MAC result, write back
ST_NEXT	Advance bk; when bk wraps, advance bc; when bc wraps, advance br
ST_DONE	Assert <code>done</code> (sticky until next <code>start</code>)

SRAM layout: element A[row][col] → address `row*9 + col`; implemented as `(row<<3) + row + col` to avoid a true multiplier. B starts at offset 81, C at offset 162.

3.4 `apb_slave.v` — APB Slave

Implements an AMBA APB v2.0 compliant slave. `pready` is tied high (zero-wait-state slave). Reads are pipelined: the SRAM address is driven combinationally during SETUP so the SRAM captures it at the rising edge; `rdata` is valid one cycle later in ACCESS.

```
// B region: paddr[9]=1 always, so word offset uses paddr[8:2]
wire sel_a = (paddr[11:9] == 3'b000) && (paddr[9:2] <= 8'd80);
wire sel_b = (paddr[11:9] == 3'b001) && ({1'b0,paddr[8:2]} <= 8'd80);
wire sel_c = (paddr[11:10] == 2'b01) && (paddr[9:2] <= 8'd80);
```

3.5 `sram.v` — Synchronous SRAM

Parameter	Value
Depth	256 words
Width	32 bits
Read	Synchronous (rdata valid 1 cycle after addr)
Write	Synchronous (write-first)

3.6 `matmul_top.v` — Top Module

Wires all modules together. Implements a simple bus arbiter: when `stat_busy=1` the tile controller owns the SRAM port; otherwise the APB slave has access. This prevents simultaneous SRAM access from both masters.

```
assign sram_we = stat_busy ? tc_we : apb_we;
assign sram_addr = stat_busy ? tc_addr : apb_addr;
assign sram_wdata = stat_busy ? tc_wdata : apb_wdata;
```

4. Systolic Array Design

4.1 Architecture Comparison

Property	Broadcast (original)	Systolic (this design)
Wiring	Global broadcast buses	Nearest-neighbour only
Scalability	Poor (wire capacitance grows)	Excellent (tile & repeat)
Tile latency	4 cycles	8 cycles (7 data + 1 clear)
Regularity	Low	High (all PEs identical)
Industry use	Small arrays	TPUs, ML accelerators

4.2 Weight-Stationary Dataflow

In weight-stationary systolic arrays, B values (weights) enter from the top and flow downward through all PE rows; A values (activations) enter from the left and flow right through all PE columns. Each PE accumulates one element of the output tile.

B[0][0]→
↓
A[0]→ PE[0][0] →
↓
A[1]→ PE[1][0] →
↓
A[2]→ PE[2][0] →

B[0][1]→
↓
PE[0][1] →
↓
PE[1][1] →
↓
PE[2][1] →

B[0][2]→
↓
PE[0][2] →
↓
PE[1][2] →
↓
PE[2][2] →

(injected top edge, flows down each cycle)

A flows right, B flows down

Each PE computes: $c_acc += a_in * b_in$ every cycle
After 7 data cycles: $PE[i][j].c_acc = \text{sum over } k \text{ of } A[i][k]*B[k][j]$ ✓

4.3 Skewing Formula

Without skewing, $A[i][k]$ would arrive at $PE[i][0]$ at different times than $B[k][j]$ at $PE[0][j]$. Diagonal skewing delays row i of A by i cycles and column j of B by j cycles:

A row i injected at cycle: $k + i + 2$
B col j injected at cycle: $k + j + 2$
They meet at $PE[i][j]$ at cycle: $k + i + j + 2$ (both arrive simultaneously ✓)
Maximum cycle needed: $k=2, i=2, j=2 \rightarrow$ cycle 8. Total data cycles = 7.

5. Tiling Strategy

5.1 Loop Order

The 9×9 multiply is decomposed into 3×3 tiles with loop order **br** → **bc** → **bk** (output-stationary / A-stationary pattern). For each output tile $C[br][bc]$, all three accumulation steps ($bk = 0, 1, 2$) are completed before moving to the next output tile.

```
for br = 0..2      // output tile row
  for bc = 0..2     // output tile col
    for bk = 0..2   // accumulation (contraction) dimension
      C[br][bc] += A[br][bk] × B[bk][bc]  // 3x3 systolic tile multiply
```

5.2 SRAM Access Pattern

Matrix	Element	SRAM Address
$A[i][j]$	Global (0..8)	$i*9 + j$ (offset 0)
$B[i][j]$	Global (0..8)	$81 + i*9 + j$
$C[i][j]$	Global (0..8)	$162 + i*9 + j$

A-tile element $A[br*3+ai][bk*3+ak]$ maps to SRAM address $(br*3+ai)*9 + (bk*3+ak)$, and similarly for B and C. The tile controller computes these addresses incrementally using the shift-add trick $(row<3) + row + col$ instead of a multiplier.

6. APB Protocol Implementation

6.1 Transaction Phases



6.2 Read Pipelining

Because the SRAM has a one-cycle read latency, the APB slave drives `sram_addr` combinationally from `paddr` during the SETUP phase. The SRAM captures the address at the SETUP→ACCESS rising edge and presents `rdata` in the ACCESS phase — exactly when the master samples `prdata`.

7. Testbench & Verification

7.1 Structure

The testbench (`tb_matmul.sv`) is written in SystemVerilog and uses automatic tasks for APB transactions. A software golden model computes the reference result and compares element-by-element against the DUT output.

Component	Description
<code>apb_write</code> task	SETUP + ACCESS phases; polls <code>pready</code> before releasing bus
<code>apb_read</code> task	Same; captures <code>prdata</code> one cycle after <code>pready</code>
<code>run_test</code> task	Loads A & B via APB, fires start, polls done, reads C, compares with ref
Golden model	64-bit signed software multiply in SystemVerilog
Timeout watchdog	250 ms wall-time limit; kills simulation if DUT hangs

7.2 Test Cases

Test	Range	Max result element	Purpose
T01	[-10 .. 10]	900	Small positive/negative mix
T02	[-20 .. 20]	3,600	Slightly wider range
T03	[-50 .. 50]	22,500	Medium values
T04	[-100 .. 100]	90,000	Stress signed arithmetic
T05	[-150 .. 150]	202,500	–
T06	[-200 .. 200]	360,000	–
T07	[-300 .. 300]	810,000	–
T08	[-500 .. 500]	2,250,000	Larger dynamic range
T09	[-750 .. 750]	5,062,500	–
T10	[-1000 .. 1000]	9,000,000	Maximum range (fits in 32-bit)

All results fit in 32-bit signed integers (2,147,483,647 max). The 64-bit internal accumulator prevents intermediate overflow during the 9 partial-product accumulations.

7.3 Simulation Results

10
TEST CASES

810
ELEMENTS VERIFIED

810
PASS

0
FAIL

GRAND TOTAL: 810 PASS | 0 FAIL

8. Project File Structure

matmul_9x9/

- rtl/
 - sram.v
 - 256x32 synchronous SRAM
 - pe.v
 - Systolic processing element (MAC + pass-through)
 - mac_array_3x3.v
 - 3x3 systolic PE array with wavefront skewing
 - tile_controller.v
 - Tiling FSM, SRAM access, partial-sum accumulation
 - apb_slave.v
 - AMBA APB v2.0 slave, address decode
 - matmul_top.v
 - Top-level: wires all modules + SRAM arbiter
- tb/
 - tb_matmul.v
 - SystemVerilog testbench, 10 randomised test cases
- setup_project.tcl
 - Creates & configures the Vivado project from scratch

9. Signed Arithmetic Notes

- All SRAM data ports are 32-bit unsigned; `$signed()` casts are applied at compute boundaries.
- PE accumulator is 64-bit signed to hold up to 9 products of 32-bit × 32-bit values without overflow.
- Tile controller partial-sum accumulation in the C region also uses 64-bit signed arithmetic internally.
- Write-back to the 32-bit SRAM truncates to the lower 32 bits — safe for all test values where results remain within $\pm 2^{31}$.
- APB read of C uses `$signed(C_dut[i][j])` in the testbench for correct negative display.

10. Key Design Parameters

Parameter	Value	Where set
Matrix dimension	9×9	Hardcoded (tiling loop bounds)
Tile size	3×3	PE array size; <code>mac_array_3x3.v</code>
Data width (DW)	32 bits	Parameter propagated from top
Accumulator width (AW)	64 bits	Parameter propagated from top
SRAM depth	256 words	<code>sram.v</code> DEPTH parameter
APB address width	12 bits	<code>paddr[11:0]</code>
Target FPGA part	xc7a35tcpg236-1	<code>setup_project.tcl</code>
Clock period	10 ns (100 MHz)	Testbench <code>always #5</code>
Tile latency	8 cycles	Systolic wavefront timing

11. Conclusion

A fully synthesizable 9×9 matrix multiplier has been designed and verified using a 3×3 systolic PE array with diagonal-wavefront skewing, a tiling FSM that drives 27 MAC operations per full multiply, an APB slave for software programmability, and a single shared SRAM for all operand and result storage.

The design passes all 810 element checks across 10 randomised test cases covering value ranges from ±10 to ±1000, confirming correct signed arithmetic at all operating points. The architecture is cleanly parameterised and can be extended to larger matrix dimensions or wider data types by adjusting DW, AW, and the tile loop bounds.

Future enhancements:

- Extend to configurable N×N dimensions via top-level parameters
- Add AXI4 interface for higher-bandwidth DMA loading
- Pipeline the tile controller to overlap LOAD and MAC phases
- Add synthesis constraints (timing, area) for Artix-7 place-and-route